

μ Agent: Orchestrating LLM Flows of Reactive and Proactive Agents on Commodity Heterogeneous SoC

OSDI'26 Submission ID: 1115

Abstract

Personal LLM agents increasingly combine foreground reactive interactions with background proactive monitoring, forming long-lived, stateful LLM flows that interleave prefill and token-by-token decode. While modern heterogeneous SoCs integrate CPUs, iGPUs, and NPUs to support on-device intelligence, existing LLM engines assume static, single-shot inference and lack mechanisms for flow-level concurrency, prioritization, and efficient accelerator coordination. As a result, commodity SoCs remain poorly matched to the dynamic, mixed-criticality execution patterns of personal agents.

This paper presents μ Agent, the first LLM engine that orchestrates concurrent reactive and proactive LLM flows on commodity SoCs. Extensive profiling uncovers unique SoC characteristics of operator-accelerator affinity, asymmetric DDR contention, and stage-divergent batching behaviors distinct from cloud-serving assumptions. μ Agent introduces three key techniques: a heterogeneous execution graph (HEG) capturing NPU/iGPU affinity and elastic operator binding; flow-aware NPU-iGPU coordination with stage elasticity, decoupling prefill and decode to reduce bandwidth contention and enforce priorities; and fine-grained preemption with slack-aware piggybacking to guarantee reactive responsiveness without starving proactive work. Across realistic personal-agent workloads, μ Agent delivers 1.2-4.9 $\times$ proactive throughput and reduces reactive latency by at least 91%, compared with both industrial iGPU-only serving engine and NPU-iGPU static inference with optimal tensor-partitioning schemes. μ Agent also minimizes energy efficiency and graphics interference via controlled iGPU usage.

1 Introduction

Large Language Models (LLMs) have revolutionized intelligent personal assistants [14], powering autonomous agentic workflows that combine answering, planning, and tool interaction [35, 62, 69]. Personal agents operate through two complementary modes [14, 15, 33, 37, 40]. As shown in Fig. 1, *reactive agents* respond to user-initiated queries in the foreground. Meanwhile, *proactive agents* continuously monitor event signals and perform speculative analysis without explicit user triggers. The agentic execution paths intersect at *LLM flows*, i.e., structured, stateful LLM invocations punctuated by stalls of human thinking or tool calling. Since LLM flows frequently

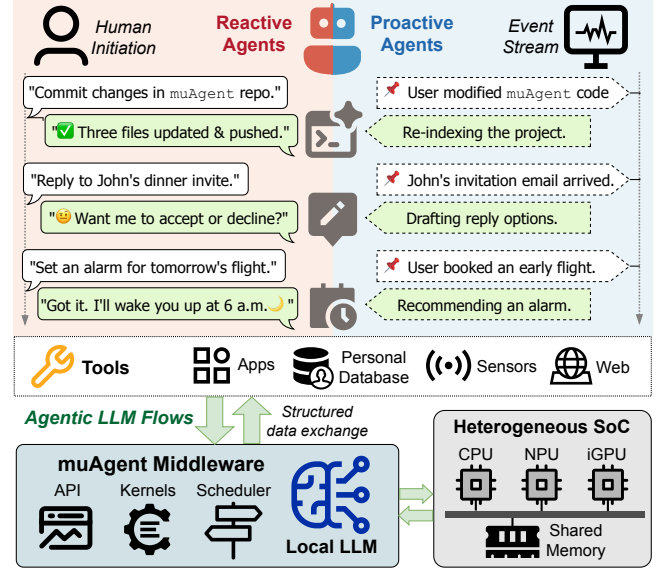


Figure 1: **Personal LLM Agent System.** μ Agent bridges the agent applications and heterogeneous SoC, orchestrating stateful on-device LLM flows from both foreground *reactive* agents and background *proactive* agents.

manipulate personal data and involve interactive steps, on-device deployment is preferable for privacy, responsiveness, and cost efficiency [66, 71, 73]. Emerging 0.6B-8B lightweight LLMs [24, 43, 44, 68] finetuned for agentic behaviors increasingly act as on-device controllers for function calls [20], with complex reasoning tasks selectively routed to cloud LLMs for effective edge-cloud collaboration [54].

Modern heterogeneous system-on-chip (hetero-SoC) across laptops and mobile phones comprises CPU, integrated GPU (iGPU), and NPU, to support local generative AI. While recent studies of on-device LLM acceleration [6, 7, 65, 67] exploit hetero-SoC via quantization or NPU offloading, they basically target *static inference*, assuming isolated, single-shot model executions with one-off prompts, non-overlapping scheduling, and uniform latency goals. In contrast, agentic LLM flows break these assumptions: they require *flow-level concurrency, coordination, and prioritization*. Reactive

flows contain real-time, bursty requests that demand timeliness; proactive flows comprise long-running, best-effort background tasks; and flow stages interleave unpredictably as agents reason or stall on external events. Current hetero-SoC hardware-software stacks offer no native support for such dynamic, flow-oriented execution, widening their mismatch with real-world personal agents.

We identify three fundamental gaps between efficient agentic LLM flows and current hetero-SoC ecosystems (§2.3).

- *Flexibility-efficiency trade-off.* NPUs deliver high performance for static-shaped input and ahead-of-time-compiled computation graphs, yet they are ill-suited for variable sequence lengths, dynamic batch sizes, and irregular control flow inherent to agentic LLM workloads. Conversely, more flexible iGPUs suffer from lower energy efficiency and potential interference with graphics responsibilities.
- *Shared-memory contention.* Concurrent agentic flows often emit overlapping NPU and iGPU kernel executions, which intensify contention for the limited DDR bandwidth of modern shared-memory SoCs, potentially degrading the latency of each individual task.
- *Absence of flow-aware runtime abstractions.* Most on-device LLM engines lack mechanisms such as in-flight batching, prioritized scheduling, and coordinated NPU-iGPU control, which are essential for launching concurrent reactive and proactive flows while meeting their distinct latency or throughput requirements.

Despite the above constraints, our in-depth hetero-SoC profiling (§3) reveals concrete opportunities to accommodate agentic LLM flows efficiently on commodity SoCs. These insights are distinct from cloud-centric LLM serving assumptions, motivating the design of our system.

- *Operator-accelerator affinity.* We observe distinct efficiency trade-offs between NPU and iGPU handling static versus dynamic LLM operators during prefill or decode, two LLM inference stages. This motivates both pre-run static placement and runtime elastic binding to map operators to their most efficient hardware backend.
- *Asymmetric contention pattern.* Memory-bound kernels are significantly more sensitive to, and more likely to saturate, shared DDR bandwidth than compute-bound kernels. This suggests that contention-aware kernel dispatching can maximize aggregate throughput by preventing concurrent memory-heavy accesses.
- *Stage-decoupled scheduling.* Co-locating all inference stages inevitably causes interference between reactive and proactive flows. However, distributing workloads based on distinct characteristics of prefill and decode across NPU and iGPU enables prioritized scheduling for reactive responsiveness while preserving proactive throughput.

- *Stage-divergent batching effects.* Prefill gains little from batching because it saturates compute regardless of batch size; decode batching improves multi-flow throughput, yet the individual latency of each flow is sensitive to total batch size and input length of all flows. This necessitates adaptive batching strategies for mixed reactive-proactive flows.

μ Agent Framework. We present μ Agent, the first LLM engine that orchestrates agentic flows on commodity hetero-SoCs (Fig. 1). μ Agent not only fills the vacuum of fully on-device concurrent serving, but aligns heterogeneous accelerators with agentic objectives by jointly optimizing reactive timeliness, proactive throughput, and energy efficiency. Its design combines profiling-driven graph construction with on-line, flow-aware NPU-iGPU co-scheduling to balance these competing goals (§4.1).

Key Techniques. First, μ Agent introduces a heterogeneous execution graph (HEG) that abstracts NPU/iGPU operator behavior in LLM flows, encoding affinity-guided placement constraints, elastic chunked kernel binding, and predictive performance annotations for runtime scheduling (§4.2). Second, μ Agent deploys flow-aware NPU-iGPU coordination with stage elasticity: prefill flows opportunistically interleave NPU and iGPU executions, while decode flows remain iGPU-resident and continuously batched. This decoupled scheduling mitigates bandwidth contention. Dedicated iGPU arbitration, prefill compute partitioning, and on-the-fly NPU kernel warm-up dynamically redistribute work under fluctuating flow concurrency (§4.3). Third, μ Agent provides fine-grained preemption for reactive queries across both stages, leveraging unified memory space for copy-free context switching (§4.4). μ Agent also utilizes slack-aware piggybacking with adaptive decode batching to prevent proactive starvation while preserving reactive responsiveness (§4.5).

Implementation and Evaluations. We implement both the agent interface and the LLM backend from scratch on Intel Core Ultra SoCs [12], representative of mainstream hetero-SoC architecture (§5). μ Agent builds atop Intel OpenVINO [47] for efficient NPU/iGPU kernel implementations and hardware-native asynchronous APIs. μ Agent is orthogonal to low-bit quantization or sparse attention and therefore preserves model accuracy.

μ Agent substantially outperforms existing on-device LLM engines (§6) across realistic personal-agent workloads (e.g., event handling, function calling, retrieval-augmented generation), using Llama3-3B/8B [43] as representative on-device LLM backbones. Baselines comprise (a) OpenVINO iGPU serving [47], (b) Llama.cpp CPU serving [22], and (c) customized *serial* NPU-iGPU inference [6, 7] with tuned tensor partitions for the target platform (§6.1). Under proactive-only workloads, μ Agent yields 1.2-2.4 $\times$ throughput over OpenVINO (iGPU) and 1.4-4.9 $\times$ over serial NPU-iGPU inference. For mixed reactive-proactive flows with diverse combinations of request heaviness, μ Agent reduces reactive query latency by 91-97% while improving proactive throughput by 0.3-58%

relative to OpenVINO (iGPU), with even larger gains over other baselines. The speedups are generally more pronounced on 8B than 3B model, indicating robust scalability (§6.2). μ Agent further reduces energy consumption by 26.8% and iGPU utilization by 32.5% compared with iGPU serving and NPU-iGPU serial baselines, respectively (§6.4).

2 Background

This section presents an overview of personal LLM agents (§2.1), the basics of LLM inference (§2.2), together with the landscape of hardware/software for on-device LLM, and their gaps towards smooth agentic flows (§2.3).

2.1 Personal LLM Agents

In this work, we focus on *personal* LLM agents deeply coupled with personal data, personal devices, and personal applications [35]. Analogous to the kernel in a traditional OS, the foundational LLM serves as the core execution engine of a personal LLM agent system, handling diverse queries issued by both reactive and proactive agents. Reactive agents are instantiated in direct response to explicit human requests. In contrast, proactive agents operate in a “human-in-the-loop” [46] manner, initiating actions autonomously based on contextual cues from the user’s environment, yet seeking human feedback or approval before execution. Mainstream LLM agent orchestration frameworks, such as LangChain [10], LlamaIndex [11], and AutoGen [63], are equipped to customize both proactive and reactive agentic workflows.

2.2 LLM Inference Primer

LLM inference is typically based on decoder-only Transformers, comprising a *prefill* stage, which encodes the prompt and generates the first token, and an auto-regressive *decode* stage to produce subsequent tokens one by one. Intermediate states (known as *KV cache* [51]) are updated after each step. The end-to-end latency of LLM inference consists of *time to first token (TTFT)*, i.e., the prefill phase, and *time per output token (TPOT)* multiplied by the number of decoded tokens after the first. In both prefill and decode, each Transformer layer constitutes three major blocks: 1) *QKV projection*, 2) *multi-head attention (MHA)*¹, and 3) *feed-forward network (FFN)*. QKV projection and FFN operate independently on each *token*, while attention attends over the entire *sequence*. This distinction underlines differences in data dependencies and parallelism between the two types of operations.

Cloud-based LLM inference is dedicated to serving user queries at high throughput while meeting service level objectives (SLOs). Established techniques include kernel optimization [13, 64], continuous batching [32, 72] with chunked

¹We use MHA as a unified term to denote attention variants such as grouped-query attention (GQA) [3] and multi-latent attention (MLA) [39].

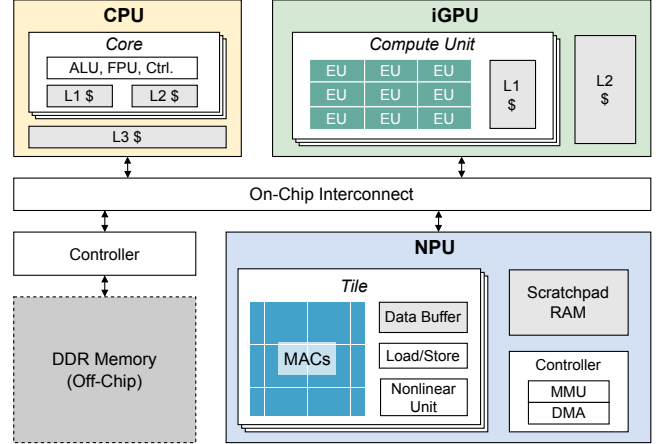


Figure 2: **Shared-Memory Hetero-SoC.** iGPU builds upon thread-level execution unit (EU), while NPU adopts multiply-accumulate (MAC) array for efficient tensor operations.

prefill [2], prefill-decode disaggregation [49, 76], KV cache reuse [1, 59, 75] or defragmentation [32], and improved tensor or pipeline parallelism [36, 57]. For resource-constrained conditions where GPU memory is limited, existing solutions propose CPU memory/disk offloading by leveraging weight locality [58] or I/O-aware scheduling [56].

2.3 On-Device LLM

Heterogeneous SoC. As illustrated in Fig. 2, hetero-SoCs exhibit a unified memory architecture distinct from heterogeneous systems with discrete accelerators: CPU, iGPU, and NPU share the same physical memory. This eliminates costly host-device data transfers but can introduce bandwidth contention caused by concurrent DDR access. Compared with cloud-hosted accelerators, commodity NPUs and iGPUs have smaller on-chip SRAM and depend heavily on shared, bandwidth-limited DRAM, requiring dedicated tuning of kernel chunking, batch dimension, and execution ordering.

From the compute angle, iGPU adopts a SIMT execution model akin to discrete GPUs, while the NPU is purpose-built for specific tensor operations, offering similar levels of parallelism but with higher energy efficiency. Nevertheless, LLMs process arbitrarily sized user inputs via dynamic-shape operators along the sequence or batch dimension, but NPUs are optimized for static-shaped operations. NPUs depend on costly compilation of fixed computational graphs to pre-allocate resources and optimize dataflows (e.g., tiling GEMMs onto fixed-size MAC arrays), making cold-start compilation at runtime infeasible. These trade-offs require NPU-iGPU collaboration to fully leverage the NPU’s compute efficiency and minimize iGPU overhead.

On-Device Inference Engines. Driven by increasing de-

Table 1: Comparison of On-Device LLM Engines.

Framework	iGPU	NPU	Hetero. Execution	Model Serving	Preempt. Support
Llama.cpp [22]	W4A16	/	CPU-iGPU	Cont. Batching	/
ONNX Runtime [18]	FP16/32	INT8	/	N/A (Offline)	/
IPEX-LLM [9]	W8A16	INT4	/	N/A (Offline)	/
MNN [42]	W4A16	/	/	N/A (Offline)	/
MLLM [65, 70]	/	INT4	/	N/A (Offline)	/
HeteroInfer [6, 7]	W4A16	W4A16	NPU-iGPU	N/A (Offline)	/
Qualcomm AI [52]	W4A16	INT4/8	/	N/A (Offline)	/
OpenVINO [47]	W8A16	INT4	/	Cont. Batching	/
μ Agent	W8A16	W8A16	NPU-iGPU	Flow-Aware Decoupling	Stage-Aware Preemption

The listed iGPU/NPU quantization methods represent the most common configurations for each framework; CPU is omitted as typically supported by default. Both W8A16 and INT8 store weights in INT8, but use FP16 and INT activation/arithmetic, respectively; similarly for W4A16 and INT4.

mands for privacy, responsiveness, and power efficiency, a range of industrial on-device LLM inference engines have emerged, including Llama.cpp [22], OpenVINO [47], ONNX Runtime [18], IPEX-LLM [9], MNN [42], QNN [19], Core ML [16], LiteRT [17], and MLLM [65, 70]. These frameworks facilitate out-of-the-box LLM deployment on vendor-specific CPUs, GPUs, or NPUs. Table 1 summarizes mainstream frameworks in comparison to μ Agent. μ Agent implements W8A16 quantization on both NPUs and GPUs to balance inference efficiency and accuracy precision, and supports stage-elastic NPU-iGPU hybrid inference. In contrast, Llama.cpp [22] enables layer-wise CPU-iGPU pipelining without temporal overlapping. Llama.cpp and OpenVINO [47] support server-client-based LLM serving with continuous batching, which is limited on singular accelerator (§ 3.2). Only μ Agent supports preemptive scheduling during each inference stage, tailored for reactive flows with sub-100 ms wait time. Recent research has explored iGPU-NPU (or CPU-NPU) co-execution via activation outlier isolation [65, 67], as well as tensor parallelism through activation or weight partitioning [6, 7]. However, these techniques primarily target reducing the end-to-end latency of monolithic inferences, whereas future on-device agentic applications demand broader support for concurrency, state dependency, and real-time interactivity.

3 Hetero-SoC Analysis and Opportunities

We conduct a comprehensive hetero-SoC analysis to guide the design of μ Agent. Operator-level analysis (§ 3.1) characterizes the compute and memory demands of representative LLM operators (ops), while task-level analysis (§ 3.2) examines runtime behavior in end-to-end agentic LLM flows. Profiling experiments are conducted on an Intel Core Ultra processor [12] with DDR5 DRAM.

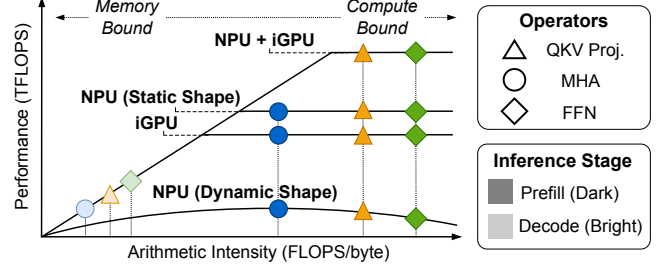


Figure 3: Schematic Roofline Illustration of LLM Ops.

3.1 Operator-Level Analysis

Operator-Accelerator Affinity. To illustrate the affinity between LLM ops and hetero-SoC accelerators, we construct a schematic roofline model (Fig. 3) derived from our profiling results of Llama [43] 3B/8B models². Static-shaped NPU kernels can be pre-compiled, whereas dynamic-shaped kernels require costly JIT compilation. For dynamic NPU prefill kernels, we amortize this cost by dividing compilation time by the number of LLM layers, as kernels are reused across layers. However, the long compilation latency makes on-demand compilation at runtime impractical. Static-shaped kernels, by contrast, offer superior energy efficiency (TFLOP-S/Watt) and comparable throughput against iGPUs. **Opportunity:** Pre-compile chunked NPU kernels for token-wise prefill ops, while offloading dynamic prefill and all decode ops to iGPU, which better handles growing dimensions and avoids NPU underutilization. At runtime, warm up NPU MHA kernels for queued requests on-the-fly to hide compilation latency and opportunistically shift prefill from iGPU.

Memory Access Pattern and Contention. Discrete GPUs with dedicated VRAM exhibit *bulk transfer and decoupled compute*, moving data between host DRAM and VRAM before or after execution. In contrast, we observe that NPUs and iGPUs with limited SRAM and DMA capacity adopt *streaming access and coupled compute*, consuming data progressively during execution, which underscores NPU-iGPU contention management under heavy DDR traffic. As shown in Fig. 4, we measure latency and bandwidth changes between separate and simultaneous GEMM/GEMV executions with Intel VTune [30]. GEMM and GEMV dominate prefill and decode stages, respectively. Co-executing memory-bound GEMV degrades both latency and bandwidth, whereas compute-bound GEMM is largely unaffected. **Opportunity:** Contention can be largely mitigated through stage decoupling of prefill and decode to NPU and iGPU (§ 3.2), complemented

²Simplifications include: arithmetic intensity of each operator varies with prompt length (here is a moderate 512 tokens); roofline curves of different ops may actually diverge and are merged here; relative NPU/iGPU peak performance are vendor-dependent; memory bandwidth slope for each curve differs by factors such as L2 or scratchpad memory size; and NPU-iGPU parallelism can slightly boost bandwidth when memory-bound (Fig. 4, [6]).

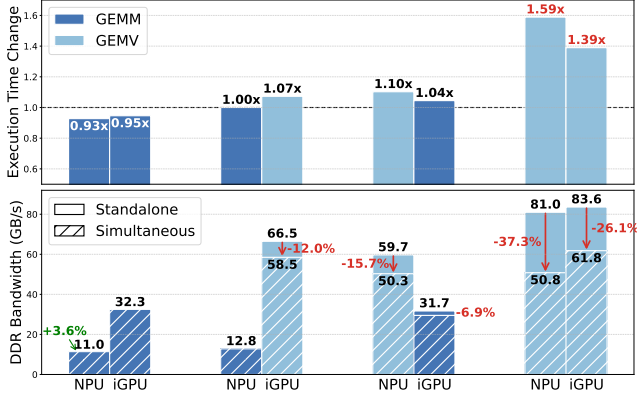


Figure 4: **Memory Contention Analysis.** Changes of execution time (upper) and DDR bandwidth (lower) from standalone NPU/iGPU kernel running to simultaneous co-execution. Memory-bound GEMV kernels are more sensitive to NPU/iGPU parallelism than compute-bound GEMM.

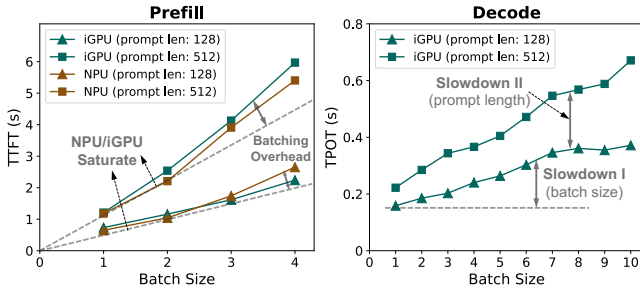


Figure 5: **Individual Task Latency in Batching.** Distinctive batching effects of prefill and decode on Llama-3B model.

by adaptive kernel dispatching with deferral (§4.3).

3.2 Task-Level Analysis

Batching Effects on Hetero-SoC. Batching intuitively improves throughput, while the individual latency is more sensitive to batch size on resource-constrained SoCs. As shown in Fig. 5, we analyze prefill and decode batching across varying batch sizes, prompt lengths, and accelerators³. Prefill latency scales nearly linearly with batch size, saturating the NPU or iGPU without performance gain. Decode latency rises gradually with larger batches or longer prompts, since each task’s MHA operates on its own KV cache with unchanged arithmetic intensity, limiting batching efficiency. This effect is amplified by constrained memory bandwidth and quadratic MHA complexity in sequence length. Furthermore, the latency gap between prefill and decode reflects decode degra-

³NPU decode is omitted since per-iteration kernel compilation is infeasible under growing sequence length and varying batch size.

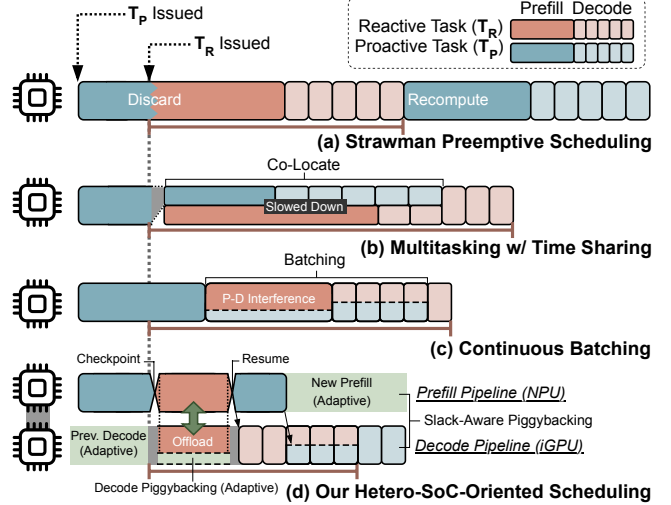


Figure 6: **Proactive-Reactive Co-Scheduling.** (a)(b)(c) target single accelerator, while (d) uses NPU and iGPU primarily for prefill and decode, respectively.

dation when batched with prefill. **Opportunity:** To trade-off latency and throughput in batching, we can disaggregate prefill and decode across NPU and iGPU (§4.3), and employ adaptive decode batching tailored to flow priorities (§4.5).

Proactive-Reactive Interference. Agentic LLM serves interleave proactive flows with real-time reactive flows, creating conflicting throughput-latency demands. Fig. 6 compares four co-scheduling schemes. Single-accelerator approaches, including (a) instant preemption without context saving, (b) time-sharing via multi-stream or virtualization, and (c) continuous batching, all suffer inefficiencies: (a) recomputation and idle time, (b) slowdowns and duplicated intermediate buffers, and (c) prefill-decode interference, which lengthens decode and is exacerbated on resource-constrained SoCs with longer prefill times. By co-locating heterogeneous stages on the same accelerator with coupled resource allocation, these methods cannot simultaneously satisfy the objectives of agentic workloads. Our hetero-SoC scheme (d) principally partitions prefill to the NPU and decode to the iGPU, with the iGPU also handling reactive prefill and dynamic prefill kernels. In this way, we achieve lowest reactive flow makespan and highest overall throughput. **Opportunity:** Minimize reactive-proactive interference via efficient preemption for reactive tasks, plus recomputation-free resumption (4.4) and slack-aware piggybacking (§4.5) guided by batching and contention profiles for proactive tasks.

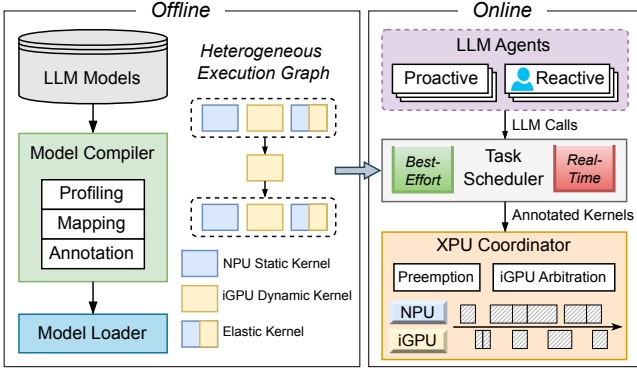


Figure 7: μ Agent System Design.

4 μ Agent Design

4.1 Contextualization and System Overview

Role of μ Agent. μ Agent serves concurrent LLM flows from personal agents, rather than handling standalone LLM inferences. Operating in a non-clairvoyant manner, μ Agent does not rely on knowledge of the agentic workflow or task arrival times. It is informed only of task priorities (proactive or reactive) at the time of issuance. μ Agent is designed with the following primary *objectives*: 1) prioritizing the reduction of end-to-end latency for LLM requests originating from reactive agents to enhance user experience, 2) increasing the overall throughput for background LLM flows from proactive agents, and 3) optimizing compute and memory resource utilization in hetero-SoC to achieve improved performance and energy efficiency.

Workload Characteristics. On resource-limited hetero-SoCs, we anticipate a typical LLM request rate from personal agents of 1 to 30 requests per minute. Calls from proactive and reactive agents are assumed to be independently distributed. LLM inferences are separate from other agentic sub-tasks, such as human interaction or tool engagement, which are assumed to be primarily CPU- or I/O-bound. Within μ Agent, iGPU usage is intentionally limited to ensure graphics availability and power efficiency.

System Design. As depicted in Fig. 7, the system comprises offline and online components. 1) *Offline*: it maintains model weights and a heterogeneous execution graph (HEG; §4.2) with pre-compiled NPU kernels and dynamic iGPU kernels. Each HEG node carries profiling-guided annotations (latency and bandwidth hints, as a function of batch/sequence size). An elastic-kernel abstraction enables late binding of each operator to NPU or iGPU at dispatch time. 2) *Online*: μ Agent dynamically schedules agentic LLM flows across the SoC on the basis of the following modules or data layouts:

① *Request Manager*. This module interfaces with the agent frontend to asynchronously admit LLM calls. It maintains

a global request table, where each entry records the request UUID, lifecycle, KV cache allocation, prompt tokens, and inference progress (phase, layer, current kernel, generated tokens). This fine-grained tracking enables task checkpointing and resuming without recomputation. The manager handles lightweight admission and moves requests across task queues.

② *Dual Task Queues*. For both prefill and decode, μ Agent maintains a real-time queue (reactive) and a best-effort queue (proactive). The queues feed the corresponding event loops, enabling efficient scheduling and immediate preemption.

③ *Prefill/Decode Event Loops*. Two dedicated loops busy-poll their queues for low-latency dispatch. They decompose tasks into NPU, iGPU, or elastic kernels and submit them to the XPU coordinator. Compute-bound prefill is run singly for each request, while decode loop adopts in-flight request batching (§4.3). The prefill loop supports sub-100 ms preemption for reactive requests at kernel boundaries (§4.4), while the decode loop dynamically adjusts batch size based on request priority and latency budgets (§4.5).

④ *XPU Coordinator*. This module orchestrates concurrent execution across NPU, iGPU, and related CPU activities (e.g., NPU kernel compilation). It tracks per-accelerator occupancy, memory residency, and contention signals, binding elastic kernels to accelerators and processing kernel-level preemption based on HEG annotations, task priority, and current load. The coordinator also arbitrates simultaneous iGPU requests under a prefill-prioritized policy (§4.3: ③).

⑤ *Recurrent Activation Double Buffer*. μ Agent maintains a single-layer activation buffer, which can be reused recurrently through layers. Prefill and decode buffers are sized by max context length and maximum batch size, respectively, with the same hidden dimension. μ Agent adopts a reactive-proactive double buffer to enable copy-free context switching for preemption (§4.4).

⑥ *Memory Manager*. To fully utilize on-device memory, μ Agent employs a background garbage collector that reclaims KV caches and on-demand NPU kernels once completed. We assume moderate request density typical of personal agents without memory overflow. Should an out-of-memory condition arise, application-directed tiering is preferred over blind paging: selectively offloading cold KV cache or weight shards to flash storage. Such offloading policies are orthogonal to our core design and can be seamlessly integrated [56].

4.2 Heterogeneous Execution Graph

Graph Representation. μ Agent models LLM execution as a heterogeneous execution graph (HEG): a parametric, accelerator-aware multigraph where nodes denote operator variants and edges capture intra- or inter-accelerator data transfers. The HEG consists of a one-shot prefill DAG and a recurrent decode micro-DAG invoked per token. Token-wise QKV projection and FFN can be chunked during prefill or

batched during decode with shared weights, while sequence-wise MHA operates per request over the individual KV cache.

Operator Mapping. To map operators onto specific accelerators while enabling runtime elasticity, we balance computational efficiency, data locality, and resource utilization:

- *Computation Affinity.* Ops are placed based on roofline characteristics (§3.1) and micro-architectural fit. For prefill, QKV projection and FFN use chunked NPU kernels or dynamic iGPU kernels; chunk sizes are tuned to saturate the NPU. Prefill MHA defaults to iGPU, though on-demand NPU warm-up can be employed. During decode, all kernels run on iGPU due to dynamic batch sizes, growing sequence lengths, and request heterogeneity. The CPU is reserved for control flow, token sampling, and kernel compilation.
- *Runtime Elasticity.* Prefill ops along the sequence dimension can be partitioned into chunked and dynamic parts. The coordinator adaptively chooses NPU or iGPU for each, depending on request priority, accelerator load, and bandwidth pressure. On-demand NPU kernels, if compiled in time, may also replace iGPU execution for dynamic ops.
- *Memory Awareness.* To minimize costly DDR transfers, we fuse consecutive ops into three core kernels, exploiting NPU/iGPU SRAM with data locality. Unlike prior approaches [6, 7, 65] that split linear and nonlinear ops across accelerators, our fused kernels exploit modern NPUs’ nonlinear units to avoid cross-accelerator transfer. For decode, we similarly fuse ops into three iGPU kernels, and further optimize by collapsing an entire decode layer into a single iGPU kernel for single-batch decode iteration.

Predictive Kernel Annotation. Combining our roofline profiling (§3.1) and analytical model (detailed in supplementary materials), we can predict kernel latency and bandwidth utilization as a function of sequence length or batch size, given accelerator choice. LLM ops are idempotent with fixed FLOPs, and we find that kernel execution times on NPU/iGPU are stable across invocations. This allows the scheduler to estimate TTFT for individual tasks and TPOT for batched decode, enabling adaptive kernel dispatching.

4.3 Flow-Aware NPU-iGPU Coordination with Stage-Elasticity

μ Agent principally decouples prefill and decode into specialized scheduling pipelines, facilitating the effective coordination of agentic flows. Prefill primarily leverages the NPU for compute-intensive token-wise ops, while decode resides on the iGPU to support dynamic batching and sequence growth. Such heterogeneous disaggregation is elastic to fit the real-time states of on-going reactive and proactive flows.

① **Prefill Pipeline.** For reactive requests, we minimize TTFT by partitioning token-wise ops across NPU and iGPU with

elastic tensor parallelism (details in ④). For proactive requests, token-wise ops run mainly on NPU, while dynamic ops (e.g., MHA) and residual fragments alongside chunked kernels run on iGPU. Because a single request already saturates either accelerator, prefill avoids batching and processes requests serially. This pipeline also serves as the substrate for the preemption mechanism of reactive requests (§4.4).

② **Decode Pipeline.** The decode pipeline adopts continuous batching decoupled from prefill, scheduling requests at iteration granularity. Reactive and proactive requests can be co-batched under adaptive strategies (§4.5). Executing all decode kernels on the iGPU naturally accommodates dynamic sequence lengths and fluctuating batch composition.

③ **Pipeline Coordination and iGPU Arbitration.** On shared memory SoCs, prefill and decode share the in-place KV cache without costly cross-accelerator transfers. However, both stages may contend for the iGPU due to common dynamic operators, necessitating arbitration mechanisms. μ Agent adopts a *prefill-first* arbitration policy: iGPU kernels from prefill always take precedence over decode, regardless of request type. This design is motivated by: 1) decode is memory-bound and generally longer than prefill, and 2) prefill requires only a small fraction of iGPU kernels, with the bulk of computation offloaded to the NPU. Prioritizing prefill ensures that short bursts of iGPU work complete promptly, avoiding long stalls that would otherwise block the entire prefill pipeline and inflate TTFT. Decode jobs can then efficiently utilize the wide gaps between prefill bursts.

④ **Elastic NPU-iGPU Tensor Parallelism.** To reduce TTFT for reactive requests while mitigating interference with on-going decode, μ Agent elastically partitions reactive prefill kernels across NPU and iGPU at runtime. Decisions are made *layer-wise* by the XPU coordinator:

1. *Decode idle:* Prefill fully exploits both NPU and iGPU, with chunked operators split so that the two accelerators complete near-simultaneously, eliminating pipeline bubbles.
2. *Decode busy:* Despite uncertain decode completion, the coordinator conservatively reduces iGPU usage by assigning fewer chunked kernels, preserving the iGPU slot for decode. Prefill iGPU kernels are deferred to complete no earlier than their parallel NPU counterparts, increasing the chance that decode finishes mid-layer with contention alleviated.
3. *Fallback:* If decode becomes idle again, prefill reverts to full NPU+iGPU parallel execution.

This elastic scheme balances reactive TTFT and decode throughput (TPOT), exploiting profiling-guided kernel annotations to align execution across heterogeneous accelerators.

⑤ **On-the-Fly NPU Kernel Warm-Up.** At runtime, μ Agent opportunistically prepares dynamic NPU kernels (e.g., MHA) once a request is queued or preempted, thereby hiding compilation latency and reducing iGPU prefill load. Since prompt

length is known at enqueue time, the CPU can start compiling static NPU kernels immediately; if compilation completes before prefill begins, the request switches to pure-NPU prefill, mitigating iGPU interference with decode. Compiled kernels are reclaimed once unused or expired. To eliminate potential contention introduced by NPU-iGPU co-execution, when NPU prefill kernels become memory-bound (e.g., MHA with short prompt length) and overlap with memory-intensive iGPU decode, the coordinator prioritizes reactive tasks: if both pipelines share the same priority (all-reactive or all-proactive), they proceed concurrently; otherwise, work with lower priority is deferred until the higher-priority side completes for reactive latency preservation.

4.4 Fine-Grained Preemption

Prioritized scheduling of reactive flows is critical for user experience. However, preemption mechanism tailored to agentic LLM workloads is absent on commodity hetero-SoCs. We exploit the unified memory and our double activation buffer design to achieve lightweight context switching. Considering fine-grained, short-lived prefill kernels and moderate reactive request frequency, we adopt lazy preemption at kernel or layer boundaries without user-perceptible delay, which avoids recompute overhead in kill-based approaches [26].

① **Copy-Free Context Switching.** An inference context comprises the KV cache, progress metadata (stage, current layer, finished kernels, generated tokens), and intermediate activations. Shared memory address eliminates the need to swap the KV cache when a reactive request arrives. Our double activation buffer separates proactive and reactive activations of prefill, enabling low-latency preemption and instant restoration without data copying.

② **Kernel-Level Prefill Preemption.** During prefill, we employ wait-then-execute preemption at kernel boundaries. Each kernel is registered with a lightweight completion callback. Upon finishing, the callback efficiently probes the reactive queue; if non-empty, it signals the prefill event loop to immediately dequeue and decompose the reactive job into kernels, thereby preempting the current proactive task. Reactive tasks do not preempt each other and run sequentially like proactive ones. Because token-wise ops are chunked, kernel execution time is tightly bounded. For an 8B model, chunked kernels (size 256) complete within 38 ms on NPU/iGPU, while MHA with sequence length 2048 completes within 97 ms. This ensures sub-100 ms preemption latency—nearly imperceptible to users with negligible TTFT impact.

③ **Iteration-Level Decode Preemption.** For decode, reactive requests wait until the current iteration completes, then immediately join the next batch. Proactive jobs may be evicted under adaptive batching policies (§ 4.5) to preserve reactive latency. Since iteration-level preemption is bounded by TPOT and the only per-request context is the localized KV cache and generated tokens, it avoids expensive swapping while still

guaranteeing fast response.

4.5 Slack-Aware Piggybacking

To preserve reactive responsiveness while preventing starvation of proactive flows, μ Agent employs slack-aware piggybacking, where proactive work opportunistically fills pipeline slack in prefill and decode with negligible impact on reactive task latency.

① **Slack Identification.** The scheduler detects two forms of slack during reactive-proactive coexistence: 1) *Compute slack*, arising from idle accelerator resources. Proactive prefill can overlap with the decode pipeline whenever no reactive prefill is pending. 2) *Bandwidth slack*, during memory-bound decode stage. Proactive requests can be co-batched with reactive ones under elaborate batching rules that bound the additional latency seen by reactive tasks, as ②.

② **Adaptive Decode Batching.** Profiling reveals that per-task decode latency increases with both batch size and request sequence length. μ Agent employs a *reactive-first* batching strategy, admitting proactive tasks only when their marginal impact on reactive latency is accepted. Each decode iteration begins by consolidating previously surviving requests with new arrivals, categorizing them into reactive and proactive groups. In homogeneous batches that are entirely reactive or proactive, all requests are included. For heterogeneous batches exceeding the predetermined maximum size B_{rf} (set to 3 in this work), proactive requests are selectively evicted to meet size constraints, prioritizing the 1) removal of longer bottleneck sequences, and 2) newly arrived tasks, to reduce TPOT and minimize system churn.

③ **Starvation Prevention.** While reactive requests receive priority, the scheduler implements aging mechanisms to prevent indefinite postponement of proactive requests. Proactive requests that exceed a threshold age are promoted to prevent starvation. They obtain two privileges without suspending reactive execution: 1) the scheduler reallocates iGPU to manage the overdue proactive prefill, while retaining reactive prefill (except MHA) solely on NPU, and 2) after prefill, the aged request can immediately join the decode batch, bypassing the batch size cap for a reactive-first batch.

5 Implementation

μ Agent exposes a RESTful API frontend (2.3K lines of Python code) for prioritized query submission in a server-client manner. The LLM backend is written in 6K lines of C++ with custom abstractions for tensors, compute graphs, and model loading, eschewing third-party dependencies (e.g., PyTorch [48], GGML [21]). This lightweight design allows μ Agent to natively embed the scheduling mechanisms introduced in § 4, while keeping kernel interfaces modular to accommodate diverse SoC platforms.

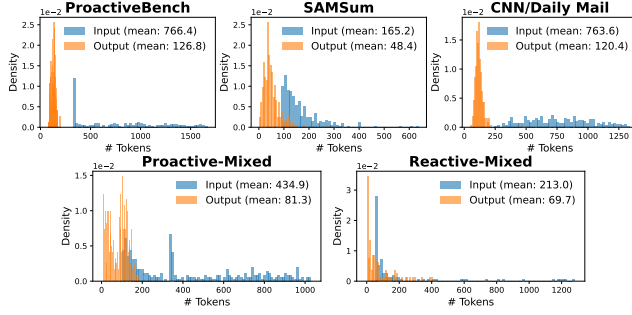


Figure 8: **Composition of Agentic Workloads.** Input/output length distributions of three proactive datasets and two proactive- or reactive-mixed datasets.

Our prototype targets Intel Core Ultra SoCs [12], implementing NPU and iGPU kernels using the low-level APIs of OpenVINO 2025.2 [47]. Request- and kernel-level scheduling is based on two-tier asynchronous interfaces: 1) *Inter-op parallelism* leverages thread-level prefill and decode event loops, coordinated via thread-safe task queues; 2) *Intra-op parallelism* exploits elastic NPU-iGPU tensor partitioning, fulfilled by the XPU coordinator with hardware-specific coroutines (`start_async/wait` in OpenVINO). The offline HEG and online scheduler jointly enable seamless adaptation to different LLM models, agent workflows, and SoC platforms.

6 Evaluation

We evaluate μ Agent on diverse agentic flow against industrial baselines (§ 6.1). Overall, μ Agent improves proactive throughput by up to $2.4\times$ over iGPU serving baseline, while reducing reactive latency by up to 97% under load (§ 6.2). Detailed breakdowns further attribute these gains to efficient heterogeneous co-scheduling (§ 6.3). We also examine the overhead management (§ 6.4).

6.1 Experimental Setup

Hetero-SoC Testbed and Models. We deploy μ Agent on an ASUS NUC 14 Pro+ mini-PC equipped with an Intel Core Ultra 5 125H processor [12] and 64GB DDR5 DRAM, running Ubuntu 24.04. The processor integrates an Intel Arc iGPU and Intel AI Boost NPU, with NPU driver v1.19.0, Intel iGPU Compute Runtime 24.39.31294.12, and performance power mode. We evaluate Llama-3.1-8B-Instruct and Llama-3.2-3B-Instruct [43] as central LLM, covering representative model architecture (GQA, dense FFN) and sizes for on-device deployment. We adopt W8A16 channel-wise quantization, incurring negligible accuracy loss [29].

Agentic Workloads. To approximate realistic personal agent behavior, we construct both proactive and reactive workloads: *Proactive*: 1) ProactiveBench [40] with real user events

such as keyboard, clipboard, and browser activity; 2) SAMSum [23], modeling group chat reply drafting; and 3) CNN/DailyMail [28], for news summarization. *Reactive*: 1) LMSys-chat-1M [74], covering diverse one-on-one conversations; 2) MTRAG [31], a multi-turn retrieval-augmented generation benchmark; and 3) Berkeley Function Call Leaderboard [50], which produces structured API calls.

We evaluate two regimes: 1) *proactive-only*: each proactive benchmark is run individually; 2) *mixed flows*: proactive and reactive requests coexist. To synthesize mixed workloads, we uniformly sample from all proactive or reactive benchmarks to form proactive- or reactive-mixed datasets. As shown in Fig. 8, the datasets exhibit remarkably different input-length distributions, reflecting the wide variation in prefill intensity across agentic tasks. Output lengths follow a long-tailed pattern, with outliers reaching up to 1.6k tokens—consistent with on-device agents producing summarized responses or compact function-call instructions, while more verbose reasoning tasks remain predominantly cloud-served.

Since the original datasets lack timestamps, we synthesize the arrival times using a Poisson process with request rates varying between 1-30 requests per minute (req/min), matching observed densities for personal agents. Proactive and reactive arrivals are generated independently. For each dataset, we evaluate the baselines and μ Agent with 15-minute traces, corresponding to increasing request rates.

Baselines. We compare against industrial on-device LLM engines with *concurrent-serving* support and optimizations for the underlying Intel SoC, as well as a customized NPU-iGPU *static-inference* baseline. All baselines use the same W8A16 precision as μ Agent.

- Llama.cpp [22], a widely used inference engine optimized for multi-core CPUs. We evaluate its serving mode with continuous batching.
- OpenVINO [47], Intel’s deployment stack for Core Ultra SoCs. Since μ Agent also builds on OpenVINO’s low-level APIs, single-batch performance is nearly identical. For serving, OpenVINO supports continuous batching only on iGPUs, which we use as the iGPU-serving baseline.
- Serial (NPU-iGPU). To generalize static tensor-parallel acceleration across NPU and iGPU [6, 7], we craft a serial pipeline that partitions prefill prompts into optimal NPU-iGPU ratios tuned for various lengths, and executes decode fully on the iGPU as NPU-iGPU parallelism barely yields decode benefit on our evaluated SoC.

Metrics. We measure both performance and efficiency metrics under varying proactive and reactive request densities: 1) *Normalized Latency*, calculated as the average request end-to-end latency divided by combined input and output lengths, gauging throughput under high request rates. We also measure P90 latency of reactive requests to highlight user-facing responsiveness; 2) *iGPU Utilization*, the weighted sum of

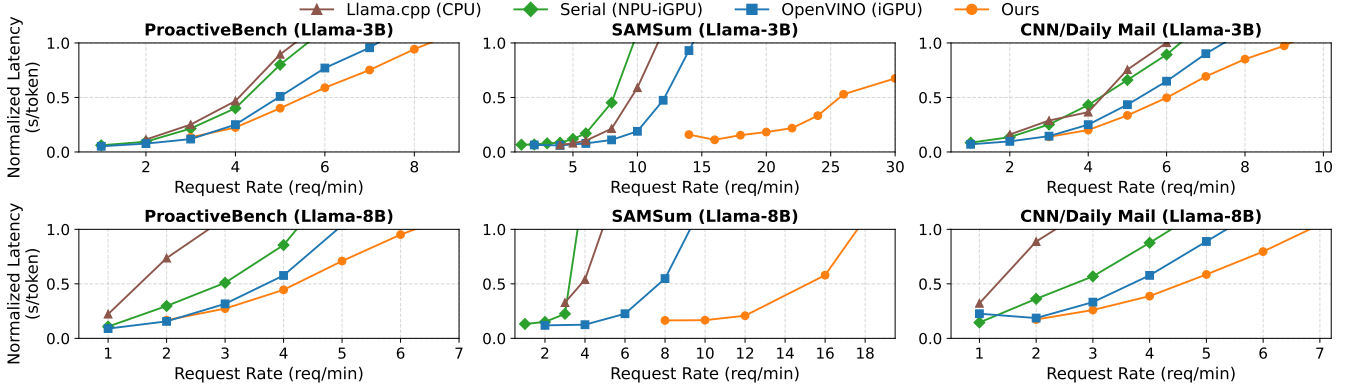


Figure 9: **Proactive-Only Workloads.** End-to-end results with Llama-3B/8B models on three proactive datasets.

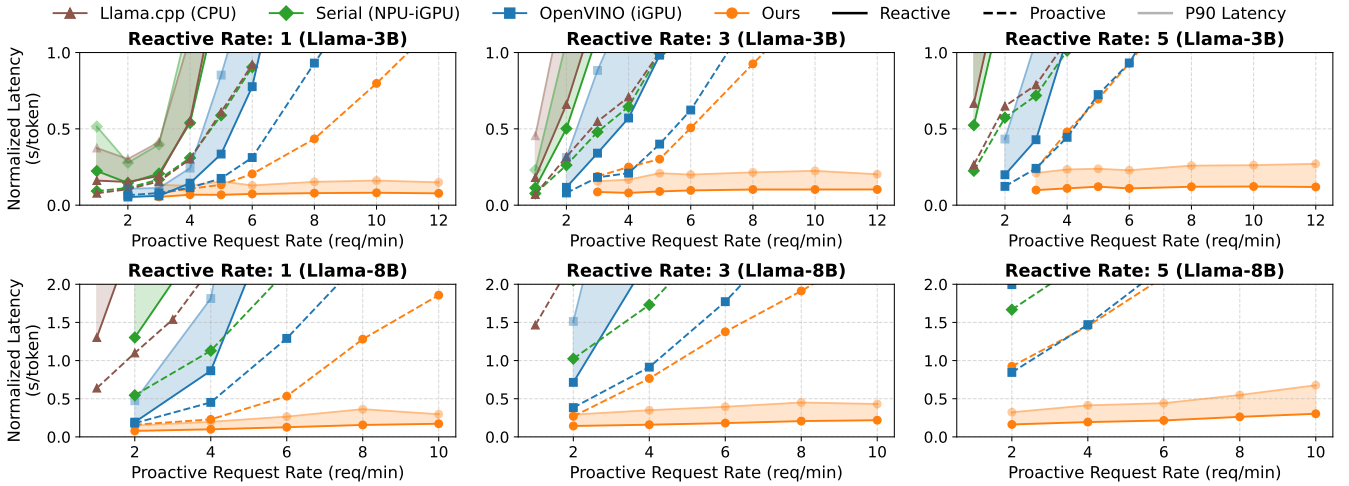


Figure 10: **Reactive-Proactive Mixed Workloads.** End-to-end results with Llama-3B/8B models on mixed datasets with varying reactive/proactive rate combinations. P90 latencies of reactive requests are displayed alongside mean latencies.

stable utilization percentage measured under distinctive loads or stages, averaged by the corresponding active execution periods; 3) *Energy per Token*, measured as the energy consumed (J/token), normalized by the processed token count.

6.2 End-to-End Performance

Proactive-Only Workloads. We first examine proactive-only workloads, where throughput is the primary metric. In this regime, μ Agent assigns uniform priority without pre-emption or adaptive batching. Although designed for mixed workloads, μ Agent already outperforms other single- or dual-accelerator baselines in throughput (Fig. 9). OpenVINO (iGPU) ranks second, benefitting from throughput-oriented continuous batching. CPU excels in decode but is bottlenecked by slow prefill, while serial NPU-iGPU execution is

limited by non-overlapping execution. μ Agent shares low-level kernels with OpenVINO with similar single-batch inference speed, but it achieves lower iGPU utilization (§6.4) by leveraging NPU-iGPU co-execution.

The performance advantage of μ Agent is further amplified under higher request rates or larger models. On SAMSUM, where prompts and outputs are relatively short, all systems sustain higher rates before saturation. With Llama-3B, μ Agent achieves 2.0-2.4 $\times$ throughput over OpenVINO (iGPU), and delivers $\sim 20\%$ more requests at 1.0 s/token latency on ProactiveBench and CNN/DailyMail. Gains are even larger with Llama-8B, as a heavier load amplifies the benefit of heterogeneous scheduling. Compared with Llama.cpp (CPU) and serial (NPU-iGPU), μ Agent achieves up to 3.9 $\times$ and 4.9 $\times$ throughput, respectively.

Reactive-Proactive Mixed Workloads. We next evaluate

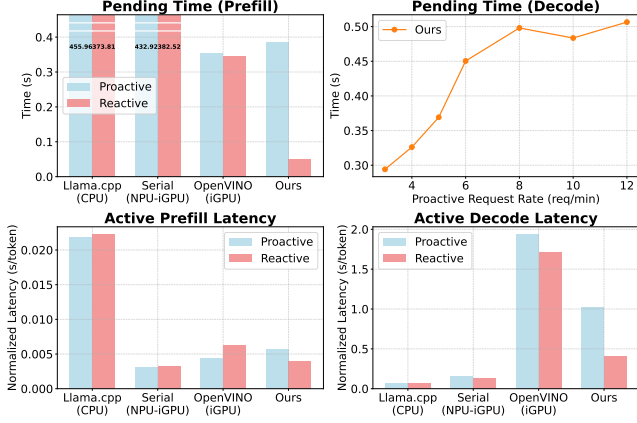


Figure 11: **Breakdown of Agentic Serving Latencies.** Most measured under mixed workloads (3 reactive and 6 proactive req/min), except the decode pending time with varying proactive rates on μ Agent, which is zero on baselines with serial scheduling or continuous batching.

mixed workloads, where proactive throughput and reactive latency must be balanced. Baselines lack request prioritization, so they treat proactive and reactive inputs equally. In contrast, μ Agent enforces reactive-first scheduling with adaptive batching. Fig. 10 reports mean and P90 latencies to capture both responsiveness and tail behavior.

μ Agent consistently achieves much lower reactive latencies while sustaining higher proactive throughput. For Llama-3B at 6 proactive req/min, μ Agent reduces mean reactive latency by 91.61%, 93.84%, and 96.01% at 1, 3, and 5 reactive req/min, respectively, compared to OpenVINO (iGPU). With Llama-8B, the reductions are 96.23%, 96.01%, and 96.70%. Tail (P90) latency reductions are even larger, underscoring improved user experience. On the proactive side, mean latency improvements reach 34.4%-0.3% for Llama-3B and 58.7%-6.1% for Llama-8B.

Reactive latency in μ Agent grows slowly with increasing proactive rates by only 19% (3B) and 49% (8B) from 4 to 10 proactive req/min, thanks to adaptive batching that caps batch sizes for reactive-first scheduling. Moreover, the gap between P90 and mean latency remains narrow, showing stable performance without outliers. In contrast, baselines degrade quickly under higher load: latencies inflate, tails widen, and responsiveness collapses. In baselines, reactive latencies consistently exceed proactive ones, as independent and irregular arrival patterns increase queuing delays under concurrency, and the absence of priority scheduling exacerbates delays for reactive requests. These results confirm the effectiveness of our co-scheduling policies.

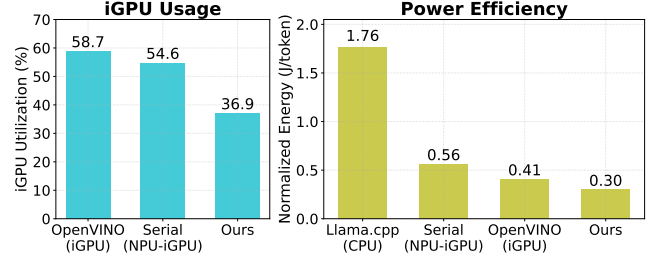


Figure 12: **Overhead Analysis.** iGPU usage and power efficiency under the same mixed workloads as Fig. 11.

6.3 Latency Breakdown

To explain the end-to-end gains, we decompose the inference latency of each request into prefill/decode pending time, active prefill latency, and active decode latency (Fig. 11). The results are normalized within either proactive or reactive agentic flows. This breakdown also serves as a compact ablation.

Baselines suffer from long prefill pending times: CPU serving is delayed by slow prefill, and static NPU-iGPU inference is stalled by serial scheduling. OpenVINO (iGPU) shows significantly longer reactive pending time than μ Agent due to the absence of effective prioritized scheduling. In contrast, μ Agent keeps reactive prefill pending time to 0.048s on average, enabled by kernel-level sub-100 ms preemption. Decode pending appears only in μ Agent due to its unique decoupled scheduling, and grows with proactive load until saturated by the maximum reactive-first batch size.

Active prefill speeds of μ Agent and OpenVINO (iGPU) are similar, but μ Agent accelerates reactive prefill via NPU-iGPU parallelism. Serial (iGPU-NPU) has uniformly better performance than μ Agent with optimal workload partition for both reactive and proactive inputs, despite its limitation to static inference. Decode shows different trends: CPU benefits from multithreading and large caches, while serial (NPU-iGPU) reflects fast single-batch iGPU decoding. iGPU decode latencies increase due to co-batching with long prefills, but μ Agent reduces reactive decode interference through reactive-first batching and fine-grained iGPU arbitration. This demonstrates why reactive requests consistently outperform proactive ones under our design.

6.4 Overhead Analysis

iGPU Utilization. We measure iGPU utilization across inference stages on the same mini PC running Windows 11, using the system resource monitor. Pure-iGPU prefill can strike 100% utilization, while decode averages 46%. The higher-than-expected decode utilization (relative to FLOPS) arises from memory traffic and synchronization overheads. We account only for active prefill and decode time, excluding duplicated computation from batching. As shown in Fig. 12,

μ Agent reduces overall iGPU utilization by 32.5% and 37.1% compared with Serial (NPU-iGPU) and OpenVINO (iGPU). Although NPU offloading reduces iGPU occupation in NPU-iGPU baseline, the serial execution without decode batching increases accumulative iGPU usage over all requests.

Power Efficiency. Using Intel VTune [30], we measure that NPU power remains stable around 10W for chunked prompt length. iGPU power grows from 25W (decode) to 31W (prefill), with little sensitivity to batch size (≤ 32). CPU workloads consume 12W during single-threaded NPU kernel compilation and up to 58W under multi-threaded inference. Fig. 12 compares per-token energy: Llama.cpp (CPU) is the least efficient, followed by Serial (NPU-iGPU) with deficient single-batch iGPU decode. μ Agent achieves 26.8% lower energy consumption than OpenVINO (iGPU) by offloading prefill to the NPU and employing efficient batching.

7 Related Work

Stateful LLM Serving. Recent systems address stateful and agentic LLM serving with caching or workflow optimizations. InferCept [1] intercepts intermediate states to avoid recomputation, while Parrot [38] and Ayo [61] expose application-level dataflows or primitive graphs for coordinated execution. Autellix [41] and SGLang [75] generalize LLM agents into program-like structures with specialized runtimes. These systems target multi-tenant cloud settings with abundant resources, whereas our focus is on single-device agents operating under tight SoC constraints.

On-Device LLM Inference. On-device inference efforts have explored heterogeneous accelerator use and architectural specialization. HeteroInfer [6, 7] and LLM.npu [65] exploit NPU-GPU (or CPU) parallelism, while PowerInfer-2 [67] adopts neuron-cluster decomposition for polymorphic execution on smartphones. These works primarily optimize isolated inference latency relying on accuracy-reducing 4-bit quantization, and lack support for stateful workloads. Hybe [45] and KTransformers [5] explore hybrid execution among server-grade accelerators with different assumptions from on-device scenarios. Orthogonal innovations in memory architecture, such as PIM-based co-design [27, 34], flexible DRAM mapping [53], and compute-enabled flash [60], require specialized hardware and limit deployability on personal devices.

Preemptive Scheduling of DNN Workloads. Preemption mechanisms improve responsiveness under concurrent deep learning workloads. PipeSwitch [4] pipelines GPU context switching, PREMA [8] predicts phases for NPU preemption, REEF [26] achieves microsecond-scale preemption via GPU reset. Pantheon [25] enables fine-grained preemption through sliced execution and early exits. XSched [55] proposes unified abstractions for XPU-based preemption. While effective for traditional DNN tasks on high-end GPUs or customized NPUs, these techniques are not applicable to the unique challenges of scheduling reactive and proactive agentic LLM

flows on hetero-SoC.

8 Conclusion

The rise of personal LLM agents demands efficient on-device execution, yet the interplay of proactive and reactive workloads remains challenging for heterogeneous SoCs. μ Agent addresses this by co-designing flow-aware scheduling with hardware heterogeneity: its heterogeneous execution graph abstraction supports elastic kernel mapping, while the online scheduler balances reactive latency and proactive throughput via fine-grained preemption, slack-aware piggybacking, and contention-aware dispatch, based on stage-decoupled NPU-iGPU pipelines. Our work paves the way for future edge platforms to serve various agentic LLM flows under tight resource constraints.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. Infercept: Efficient intercept support for augmented large language model inference. In *International Conference on Machine Learning*, pages 81–95, Vienna, Austria, 2024. JMLR.org.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, Santa Clara, CA, USA, 2024. USENIX Association.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.
- [4] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. {PipeSwitch}: Fast pipelined context switching for deep learning applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 499–514, 2020.
- [5] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, et al. Ktransformers: Unleashing the full potential of cpu/gpu hybrid inference for moe models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, pages 1014–1029, 2025.

- [6] Le Chen, Dahu Feng, Erhu Feng, Yingrui Wang, Rong Zhao, Yubin Xia, Pinjie Xu, and Haibo Chen. Characterizing mobile soc for accelerating heterogeneous llm inference. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, pages 359–374, 2025.
- [7] Le Chen, Dahu Feng, Erhu Feng, Rong Zhao, Yingrui Wang, Yubin Xia, Haibo Chen, and Pinjie Xu. Heterollm: Accelerating large language model inference on mobile socs platform with heterogeneous ai accelerators. *arXiv preprint arXiv:2501.14794*, 2025.
- [8] Yujeong Choi and Minsoo Rhu. Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 220–233. IEEE, 2020.
- [9] IPEX-LLM contributors. IpeX-llm toolkit. <https://github.com/intel/ipex-llm>, 2024.
- [10] LangChain contributors. Langchain: A composable framework to build with llms. <https://www.langchain.com>, 2023.
- [11] LlamaIndex contributors. Llamaindex: Build ai knowledge assistants over your enterprise data. <https://www.llamaindex.ai>, 2023.
- [12] Intel Corporation. Intel core ultra processors. <https://www.intel.com/content/www/us/en/products/details/processors/core-ultra.html>, 2024.
- [13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- [14] Allan de Barcelos Silva, Marcio Miguel Gomes, Cristiano André Da Costa, Rodrigo da Rosa Righi, Jorge Luis Victoria Barbosa, Gustavo Pessin, Geert De Doncker, and Gustavo Federizzi. Intelligent personal assistants: A systematic literature review. *Expert Systems with Applications*, 147:113193, 2020.
- [15] Yang Deng, Lizi Liao, Zhonghua Zheng, Grace Hui Yang, and Tat-Seng Chua. Towards human-centered proactive conversational agents. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 807–818, Washington DC USA, 2024. ACM.
- [16] Core ML developers. Core ml - integrate machine learning models into your app. <https://developer.apple.com/documentation/coreml/>, 2023.
- [17] LiteRT developers. Google litert overview. <https://ai.google.dev/edge/litert>, 2025.
- [18] ONNX Runtime developers. Onnx runtime. <https://onnxruntime.ai/>, 2021.
- [19] QNN developers. Qualcomm ai engine direct (qnn). <https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-50/overview.html>, 2025.
- [20] Lutfi Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Ghلامي. Tinyagent: Function calling at the edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 80–88, Miami, Florida, USA, 2024. ACL.
- [21] Georgi Gerganov. Ggml tensor library for machine learning. <https://github.com/ggml-org/ggml>, 2023.
- [22] Georgi Gerganov. llama.cpp - inference of meta’s llama model (and others) in pure c/c++. <https://github.com/ggml-org/llama.cpp>, 2023.
- [23] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. Samsun corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, pages 70–79, Hong Kong, China, 2019. Association for Computational Linguistics.
- [24] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [25] Lixiang Han, Zimu Zhou, and Zhenjiang Li. Pantheon: Preemptible multi-dnn inference on mobile edge gpus. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services*, pages 465–478, 2024.
- [26] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent {GPU-accelerated}{DNN} inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 539–558, 2022.
- [27] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 722–737, 2024.

- [28] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend, 2015.
- [29] Wei Huang, Xingyu Zheng, Xudong Ma, Haotong Qin, Chengtao Lv, Hong Chen, Jie Luo, Xiaojuan Qi, Xianglong Liu, and Michele Magno. An empirical study of llama3 quantization: from llms to mllms. *Visual Intelligence*, 2(1):36:1–36:13, December 2024.
- [30] Intel. Intel® vtune™ profiler. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html#gs.i6xhgk>, 2025.
- [31] Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. Mtrag: A multi-turn conversational benchmark for evaluating retrieval-augmented generation systems. *Transactions of the Association for Computational Linguistics*, 13:784–808, 2025.
- [32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with page-dattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, Koblenz Germany, 2023. ACM.
- [33] LangChain. Introducing ambient agents. <https://blog.langchain.dev/introducing-ambient-agents/>, 2025.
- [34] Cong Li, Yihan Yin, Xintong Wu, Jingchen Zhu, Zhutianya Gao, Dimin Niu, Qiang Wu, Xin Si, Yuan Xie, Chen Zhang, et al. H2-llm: Hardware-dataflow co-exploration for heterogeneous hybrid-bonding-based low-batch llm inference. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, pages 194–210, 2025.
- [35] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.
- [36] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 663–679, Boston, MA, 2023. USENIX Association.
- [37] Lizi Liao, Grace Hui Yang, and Chirag Shah. Proactive conversational agents in the post-chatgpt world. In *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*, pages 3452–3455, 2023.
- [38] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, Santa Clara, CA, USA, 2024. USENIX Association.
- [39] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [40] Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, et al. Proactive agent: Shifting llm agents from reactive responses to active assistance. In *The Thirteenth International Conference on Learning Representations*, 2024.
- [41] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- [42] Chengfei Lv, Chaoyue Niu, Renjie Gu, Xiaotang Jiang, Zhaode Wang, Bin Liu, Ziqi Wu, Qiulin Yao, Congyu Huang, Panos Huang, et al. Walle: An End-to-End, General-Purpose, and Large-Scale production system for Device-Cloud collaborative machine learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 249–265, 2022.
- [43] Meta. Llama 3.2. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>, 2024.
- [44] Microsoft. Phi-4-mini-instruct. <https://github.com/marketplace/models/azureml/Phi-4-mini-instruct>, 2024.
- [45] Seungjae Moon, Junseo Cha, Hyunjun Park, and Joo-Young Kim. Hybe: Gpu-npu hybrid system for efficient llm inference with million-token context window. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, pages 808–820, 2025.

- [46] Eduardo Mosqueira-Rey, Elena Hernández-Pereira, David Alonso-Ríos, José Bobes-Bascarán, and Ángel Fernández-Leal. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*, 56(4):3005–3054, 2023.
- [47] OpenVINO. Open-source toolkit for deploying performant ai solutions in the cloud, on-prem, and on the edge alike. <https://docs.openvino.ai/2025/index.html>, 2018.
- [48] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.
- [49] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132, Buenos Aires, Argentina, 2024. IEEE.
- [50] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- [51] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5:606–624, 2023.
- [52] Qualcomm. Deploy llama-v3.2-3b-instruct on snapdragon 8 elite mobile. https://aihub.qualcomm.com/mobile/models/llama_v3_2_3b_instruct, 2024.
- [53] Seong Hoon Seo, Junghoon Kim, Donghyun Lee, Seonah Yoo, Seokwon Moon, Yeonhong Park, and Jae W Lee. Facil: Flexible dram address mapping for soc-pim cooperative on-device llm inference. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1720–1733, Las Vegas, NV, USA, 2025. IEEE.
- [54] Chenyang Shao, Xinyuan Hu, Yutang Lin, and Fengli Xu. Division-of-thoughts: Harnessing hybrid language model synergy for efficient on-device agents. In *Proceedings of the ACM on Web Conference 2025*, pages 1822–1833, 2025.
- [55] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. {XSched}: Preemptive scheduling for diverse {XPU}s}. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 671–692, 2025.
- [56] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116, Honolulu, HI, USA, 2023. PMLR.
- [57] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [58] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, pages 590–606, Austin, TX, USA, 2024. ACM.
- [59] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for LLM serving. In *The Thirteenth International Conference on Learning Representations*, Singapore, 2025. openreview.net.
- [60] Weiyi Sun, Mingyu Gao, Zhaoshi Li, Aoyang Zhang, Iris Ying Chou, Jianfeng Zhu, Shaojun Wei, and Leibo Liu. Lincoln: Real-time 50~100b llm inference on consumer devices with lpddr-interfaced, compute-enabled flash memory. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1734–1750, Las Vegas, NV, USA, 2025. IEEE.
- [61] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1302–1316, Rotterdam, Netherlands, 2025. ACM.
- [62] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [63] Qingyun Wu, Gagan Bansal, Jiayu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.

- [64] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099, Honolulu, HI, USA, 2023. PMLR, JMLR.org.
- [65] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. Fast on-device llm inference with npus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 445–462, Rotterdam, Netherlands, 2025. ACM.
- [66] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088*, 2024.
- [67] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024.
- [68] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [69] Ke Yang, Jiateng Liu, John Wu, Chaoqi Yang, Yi Rong, Sha Li, Zixuan Huang, Xu Cao, Xingyao Wang, Yiquan Wang, et al. If llm is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents. *arXiv preprint arXiv:2401.00812*, 2024.
- [70] Rongjie Yi, Xiang Li, Zhenyan Lu, Hao Zhang, Daliang Xu, Liming Yang, Weikai Xie, Chenghua Wang, Xuanzhe Liu, and Mengwei Xu. mllm: fast and lightweight multimodal llm inference engine for mobile and edge devices. <https://github.com/UbiquitousLearning/mllm>, 2023.
- [71] Wangsong Yin, Mengwei Xu, Yuanchun Li, and Xuanzhe Liu. Llm as a system service on mobile devices. *arXiv preprint arXiv:2403.11805*, 2024.
- [72] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. ORCA: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, USA, 2022. USENIX Association.
- [73] Jinliang Yuan, Chen Yang, Dongqi Cai, Shihe Wang, Xin Yuan, Zeling Zhang, Xiang Li, Dingge Zhang, Hanzi Mei, Xianqing Jia, et al. Mobile foundation model as firmware. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 279–295, Washington D.C., DC, USA, 2024. ACM.
- [74] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P Xing, et al. Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998*, 2023.
- [75] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37:62557–62583, 2024.
- [76] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, USA, 2024. USENIX Association.